

Morlet wavelets in time and in frequency?



In time domain, we use a gaussian to create a wavelet, in frequency domain, the amplitude spectrum is a gaussian

we can also create morlet wavelet in time domain, by creating a gaussian in frequency domain and taking inverse fourier transform of that gaussian will also give us morlet wavelet in time domain

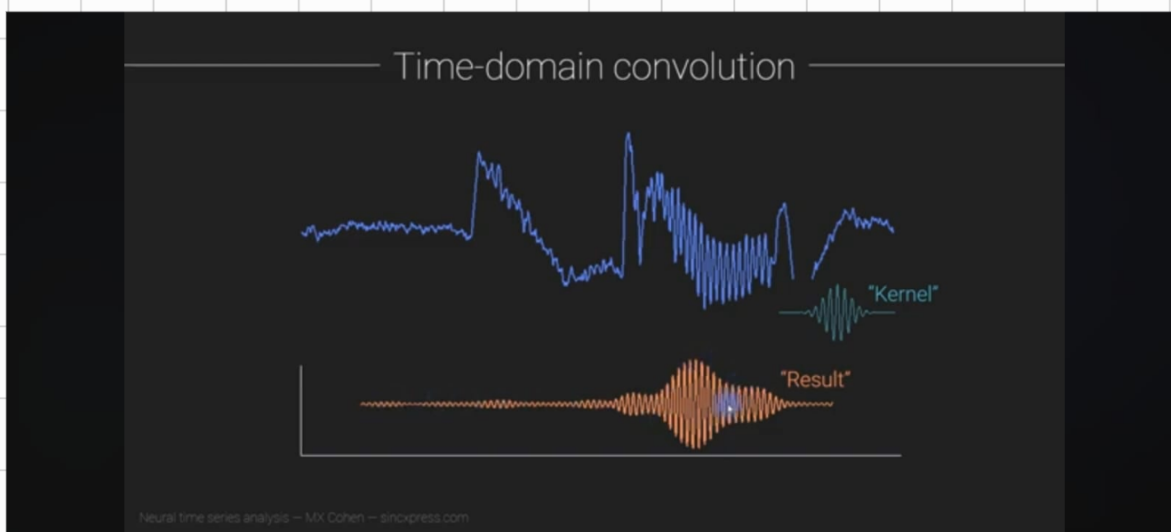
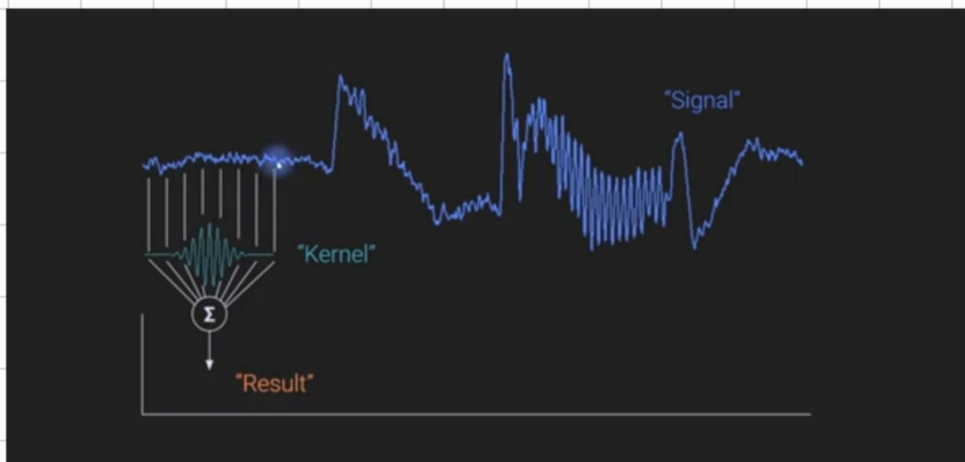
Convolution in the time domain:

we should avoid time domain convolution whenever possible because it's slow, and it requires a lot of extra overhead/extra code

goal of convolution: we take two time series and somehow mix them together to create a third time series

Reason why we will use complex morlet wavelets instead of real valued wavelet

↳ sign dependency of Convolution



No. of points we loose at start and end of Convolution depends on size of Kernel

To avoid losing points, we start convolution by aligning the rightmost point of the Kernel with the leftmost point of data, that won't be valid dot product operation, so we will pad with zeroes

Zero padding in Convolution is different from zero padding in Fourier transform:

In Fourier transform, we will put all zeroes at the end $\rightarrow$ to increase the frequency resolution

$\rightarrow$ At the end, we will be having leftmost point of the Kernel aligned with rightmost point of the signal, so here also not valid dot product, so pad with zeroes

length of the result of Convolution = $n + m - 1$

length of signal $\downarrow$ length of Kernel

Kernel flipping: $\rightarrow$ doesn't need to worry about this in freq domain, it only happens in time domain

Kernel flipping

Actual kernel

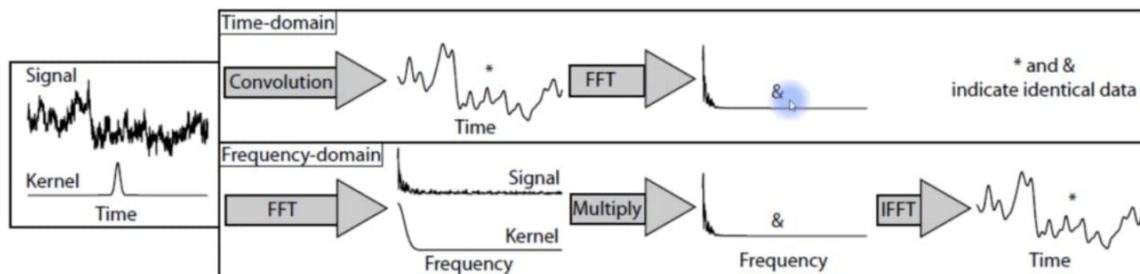


Kernel in convolution



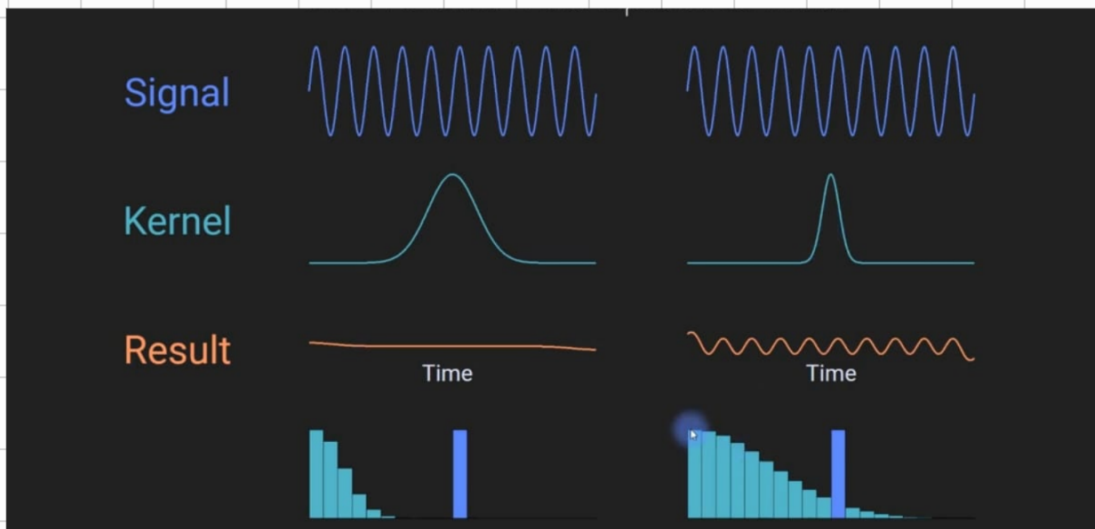
the Convolution theorem:

convolution in the time domain is multiplication
in the frequency domain



Convolution in the time domain is
multiplication in the frequency domain.

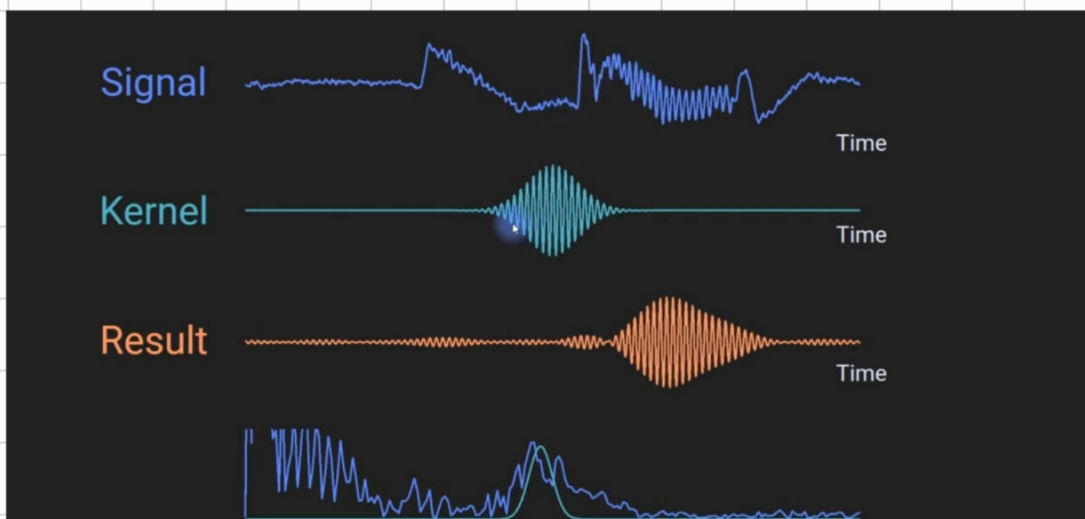
Convolution as spectral filter:



In the above image, interpreting Convolution in time domain is not that easy, we can understand that in left half, since kernel is wide, most of oscillations comes into kernel and cancel each other, but that won't be case in right side $\rightarrow$ This is for time domain but frequency domain is easier

In frequency domain, result of convolution is going to be features of signal that share characteristics with features of kernel

Convolution as Spectral filter for previously discussed example:



After just multiplying overlapping frequencies in FFT's and just stopping $\rightarrow$ there will be no advantage of this approach wavelet convolution over the static fourier transform

because still when we are here in frequency domain, we still don't have any info about temporal dynamics / Non stationarity

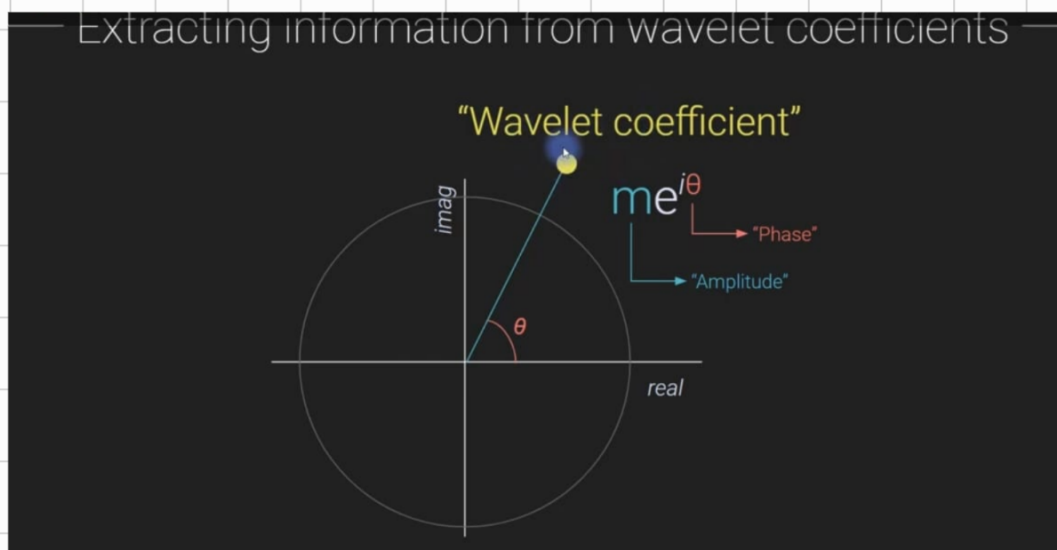
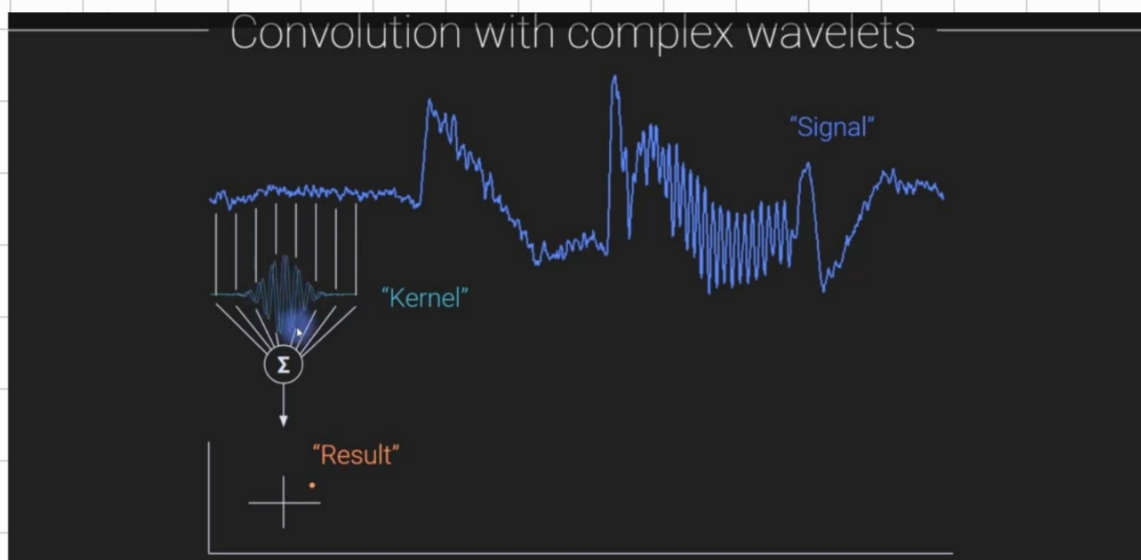
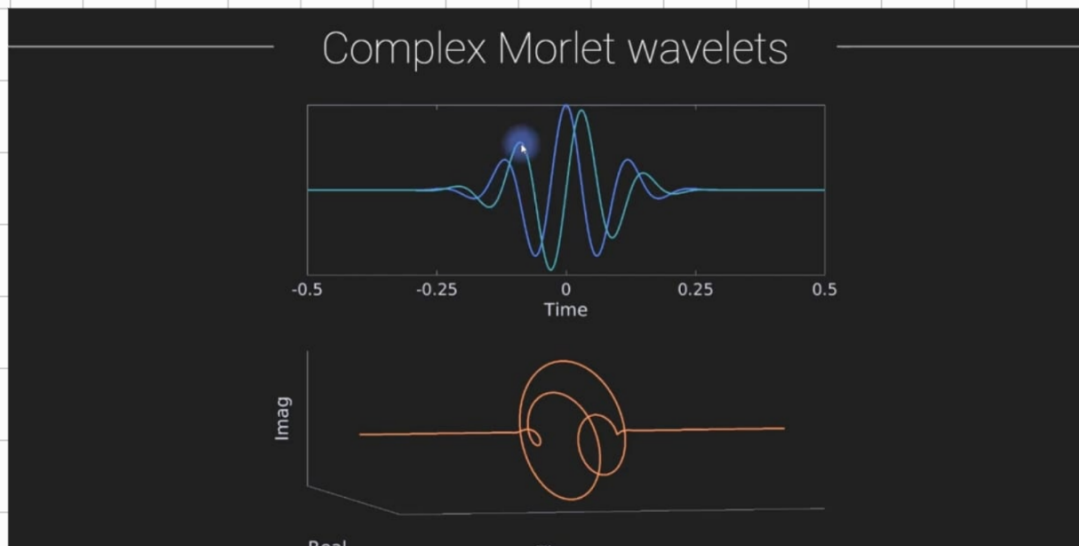
After multiplying, we should take IFT to get back into time domain

↳ Here we get the spectral information isolated from the frequency domain multiplication and then IFT gives us the temporal dynamics, but only the temporal dynamics that are in the frequency range specified by / constrained by Kernel

Five steps of Convolution:

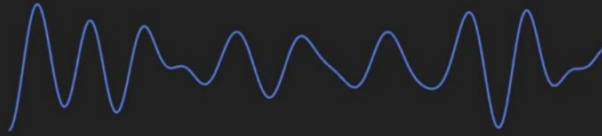
- 1) Compute N 's $\rightarrow$ no. of time points in signal,
no. of time points in kernel;
length of result of convolution
- 2) FFTs of signal & kernel
- 3) multiply spectra element by element
- 4) IFFT
- 5) cut of wings

Complex Morlet wavelet Convolution:

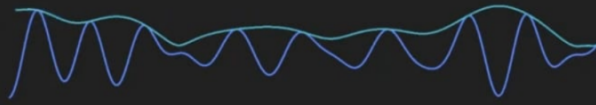


Terminology of time-frequency features

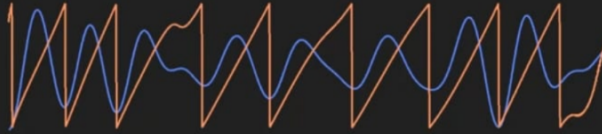
Narrowband
signal



Amplitude
time series

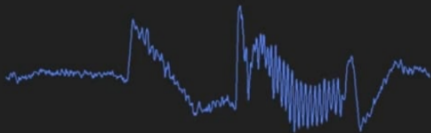


Phase angle
time series

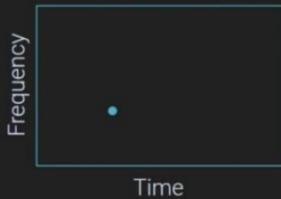


Neural time series analysis — MX Cohen — sincxpress

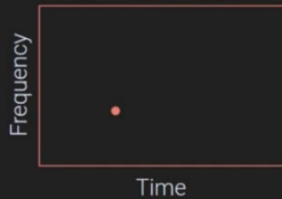
Extracting information from wavelet coefficients



Time-frequency power



Time-frequency phase



Narrowband filtered signal



→ we will use complex morselet wavelets because of same reason as using complex sine wave in fourier transform, that is because of coupling b/w power & phase

Multi trial Convolution trick:

```
for chan = 1 : numchans  
    for freq = 1 : numfreq
```

```
        for trial = 1 : numtrials
```

Convolution FFT's and IFFT's

```
        end
```

```
    end
```

```
end
```

→ To get rid of this loop, we will concatenate all trials into single one and now apply Convolution then we can extract whatever we want to like power time series, phase angle time series, bandpass filtered signal and then we can split this back up into individual trials

```
for chan = 1 : numchans
```

fourier transform of data

```
for freq = 1 : numfreq
```

fourier transform of wavelet
doing spectral multiplication
inverse FFT

```
end
```

```
end
```


Parameters of Morlet wavelet (Time-Frequency tradeoff):

Morlet wavelet key parameter:

$$e^{i2\pi ft - 0.5(t/\sigma)^2}$$

normalisation factor

$$\sigma = \frac{\eta}{2\pi f}$$

no. of cycles

$$-\frac{(t/s)^2}{\frac{e}{2}}$$

same freq used for sine wave

the standard way:

$$w = e^{i2\pi ft} \cdot e^{-t^2/2\sigma^2}$$

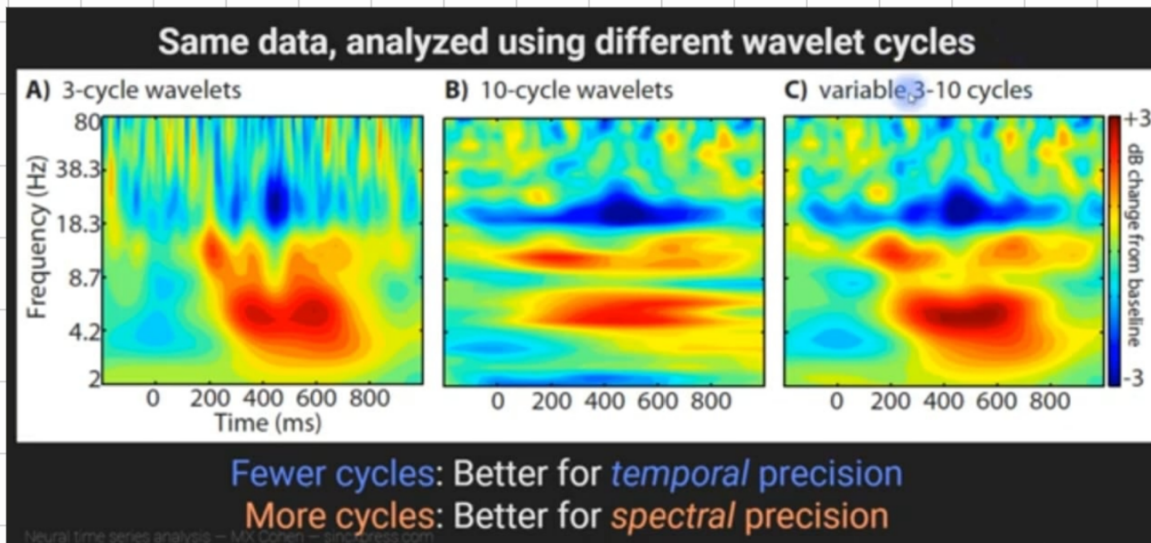
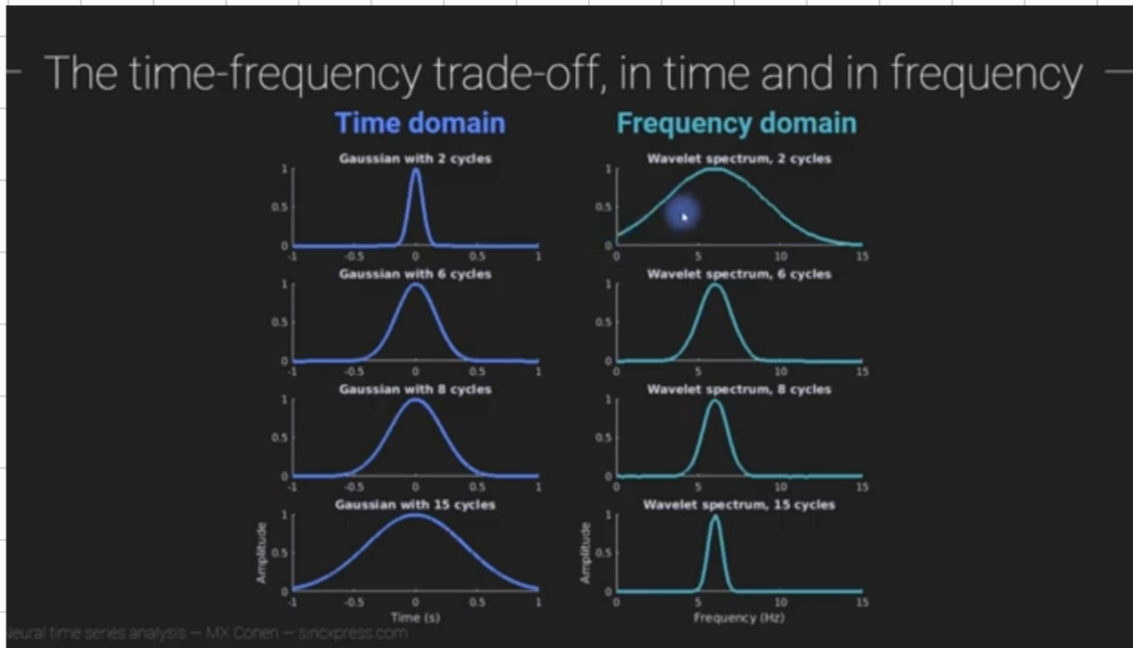
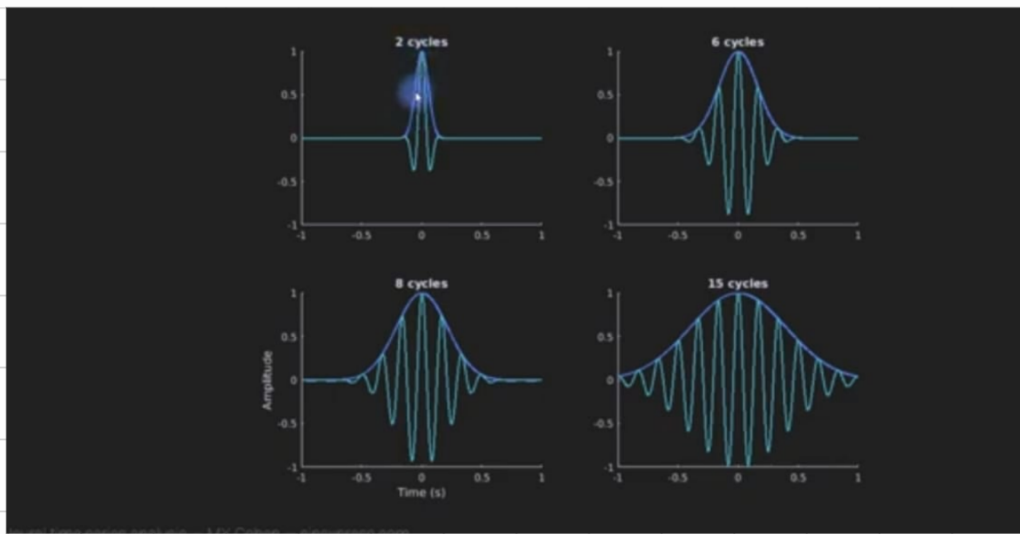
$$\sigma = \frac{\eta}{2\pi f}$$

$$w = e^{i2\pi ft} \cdot e^{-\frac{4\ln(2)t^2}{h^2}}$$

As the no. of cycles increases $\rightarrow$ the gaussian gets wider
the wavelet energy is wider in time

low no. of cycles $\rightarrow$ temporal precision of that wavelet will be high

$\hookrightarrow$ we can have lot of confidence in the localization of our frequency specific features to a specific time point



Here in first plot, we can see two red circles at lower frequencies → so better temporal precision, but we can't see diff b/w lower & higher freq b/w frequency → so not better for spectral precision

In 2nd, we can't see distinction b/w diff times so No temporal precision

but we can say distinction b/w spectrums separated $\rightarrow$ so spectral precision.

The stationarity assumption of wavelet convolution:

The results of the Fourier transform are easily interpretable only for stationary signals

The results of wavelet Convolution are easily interpretable only for stationary signals within FWHM (Full width Half maximum) of the wavelet, where the wavelet has Non Zero energy.

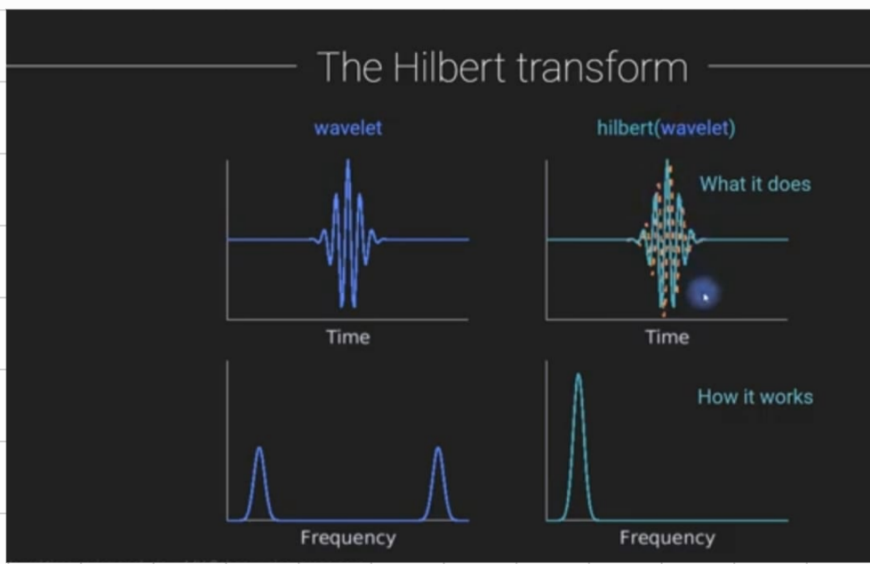
The filter-Hilbert time frequency method:

Hilbert transform:

When we take Hilbert transform of real valued signal, output is complex valued signal

Mechanism of Hilbert transform:

- 1) Take the FFT of signal
- 2) Go into the frequency domain and zero out all of the negative frequencies, double the amplitudes of positive frequencies
- 3) And then take the Inverse Fourier transform



The goal of Hilbert transform is not to give us complex valued wavelet, but instead it is to give us a complex valued signal also called an analytic signal that we can use to extract power/Amplitude and phase information in addition to the real part of the signal

Application of the Hilbert Transform:

- Convert a real valued signal into a complex valued signal
- The result is analogous to the result of complex morselet wavelet convolution
- we can then extract magnitude & phase just like you would with the result of complex morselet wavelet convolution

Imp point :

- The power and phase at each time point resulting from hilbert transform come from the frequency with the most power at that time point

→ The output of the Hilbert transform is interpretable only for Narrowband signals

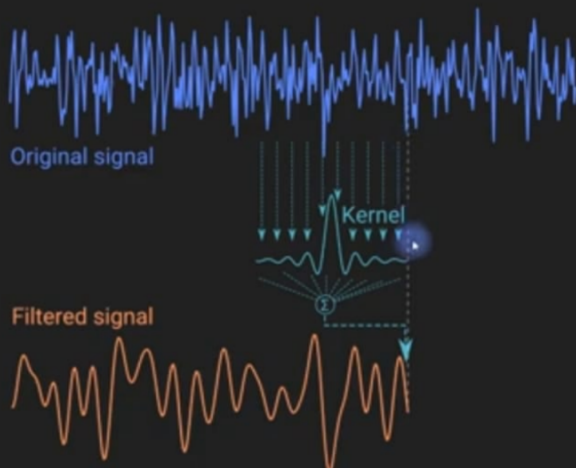
→ Do not apply the Hilbert transform to broadband data

→ EEG/LFG are broadband → they have energy at whole range of frequencies

What's the solution?

wavelet convolution → Narrowband filtering with gaussian

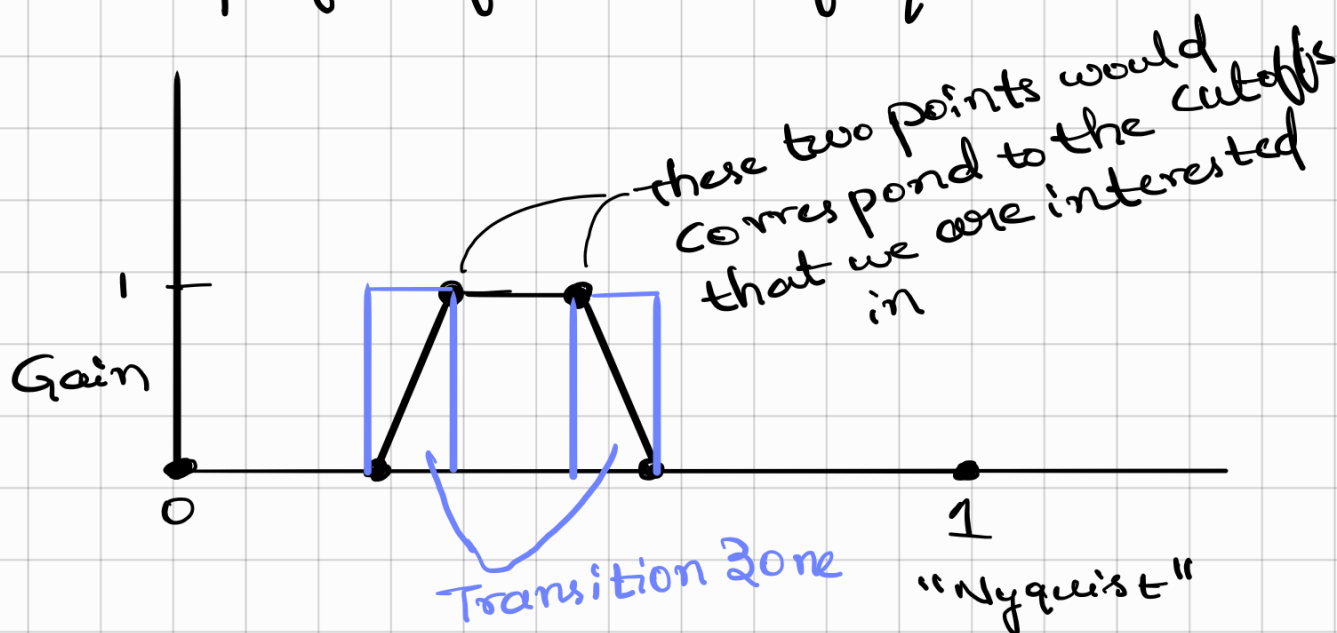
FIR filters for narrowband isolation



Designing FIR Bandpass filters:

we want filters to be able to be designed without having to worry about the sampling rate, that's why we only specify in terms of Nyquist

we have four data points that define the shape of the filter in the freq domain



The idea of having transition zone is that we don't want to specify the filter with perfect edges, we don't want a sharp edge in the freq domain and the reason is that will introduce sharp edges in freq domain, when we take IFT, that sharp edge going to produce a lot of ripple effects in time domain, which can introduce artifactual oscillations into our data

Transition zone will be typically 10% or
of 15% of that point
output will be filter kernel in time domain
↳ gets applied to the
time series data and
that gives us narrow
band signal

Let signal that we get back is going
to be a real valued narrowband
signal & then we would apply hilbert
transform to the output of the
filter

we can increase freq resolution by decreasing
no. of timepoints
↳ called as order/
taps

^{of filter}
The more time points we have, the
better the freq domain response is

once IIR filters gets longer → they will
become unstable

FIR filters are highly stable → they give
higher quality results, particu-
larly at estimating phase values

IIR filters are less robust, they are more
sensitive to noise/other unusual features in

data

filter fast $\rightarrow$ IIR $\rightarrow$ but less accurate

reason, FIR involve multiplying data with the kernel, so we get our kernel and multiply it by data $\rightarrow$ and this gives result

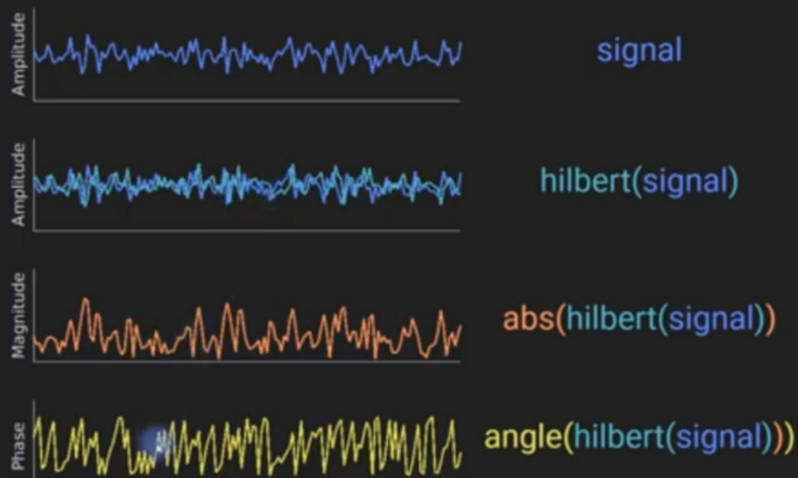
IIR works as we take our short little kernel and we filter a little piece of data and then we take the data that we have just filtered and use that as an additional part of the kernel for filtering even more data $\rightarrow$ so we are using data to filter more data

$\hookrightarrow$ If weird things happening in data, that can introduce artifacts into kernel, strange things happening in data are going to reverberate further down the data and get worse & worse over time

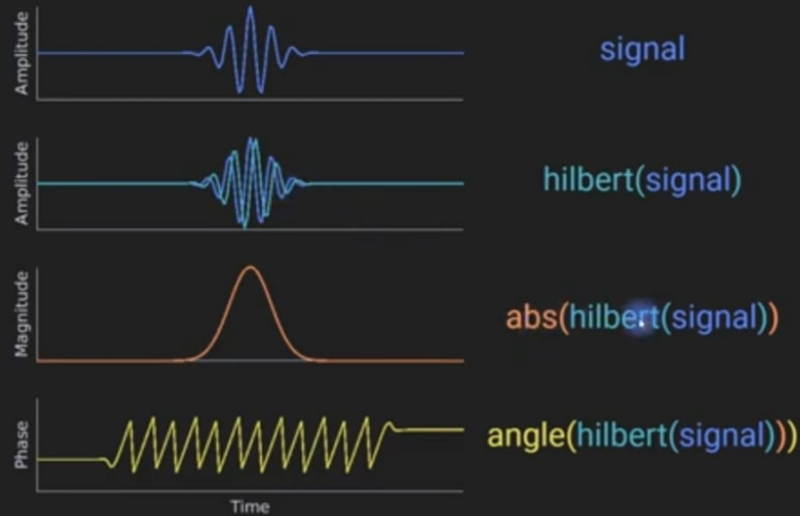
How to implement the filter hilbert method:

- 1) filter
- 2) Hilbert
- 3) Extract power and/or phase

The Hilbert transform: example with noise



The Hilbert transform: example with wavelet



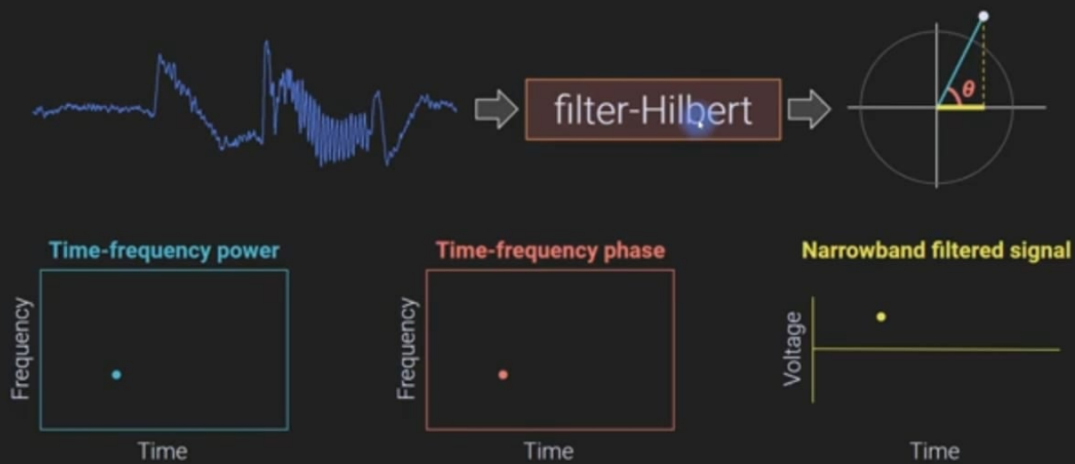
Neural time series analysis — MX Cohen — sinoxpress.com

FIR vs. IIR filters

	FIR	IIR
Name	Finite impulse response	Infinite impulse response
Kernel length	Long	Short
Speed	Slower	Fast
Stability	High	Data-dependent
Mechanism	Multiply data with kernel	Multiply data with data

Neural time series analysis — MX Cohen — sinoxpress.com

Extracting information from filter-Hilbert



Neural time series analysis — Mx Cohen — sincpress.com

Another method for doing Time Freq Analysis is

↳ Short-time Fourier transform

STFT/STFFT

↳ How can we adapt Fourier transform procedure so that instead of weighting all timepoints equally we can focus analysis on one specific time window.

↳ Instead of doing Fourier transform on entire signal, we only do the Fourier transform on one little segment of the signal → this gives us the spectral characteristics of that window of time and we do for all windows

STFFT time-frequency precision tradeoff:

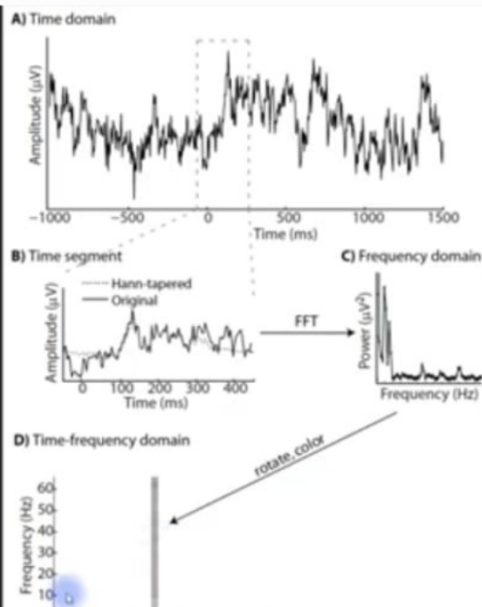
size of the window is going to determine temporal precision, spectral resolution
time windows $\rightarrow$ Narrow, don't have lot of freqs b/w 0 and Nyquist

$\rightarrow$ longer, more frequencies
frequency resolution increases, temporal precision decrease

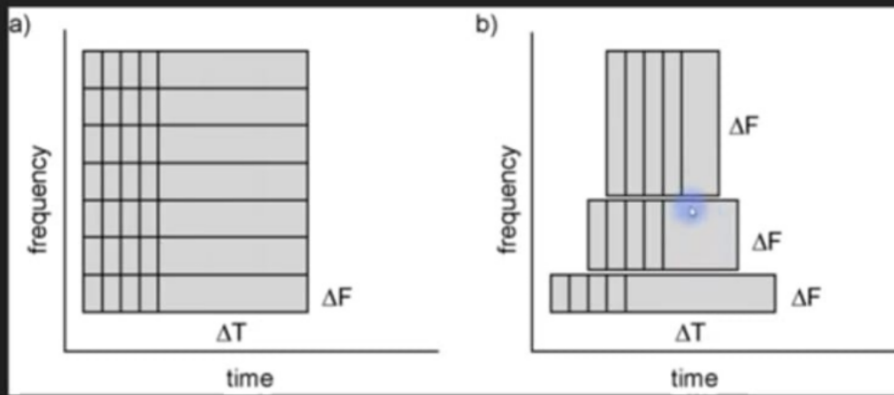
width of window
amount of overlap b/w successive windows $\rightarrow$ more overlap, smoother
time freq plot, computation time increases

way to change the size of time windows with diff frequencies

Mechanism of STFFT



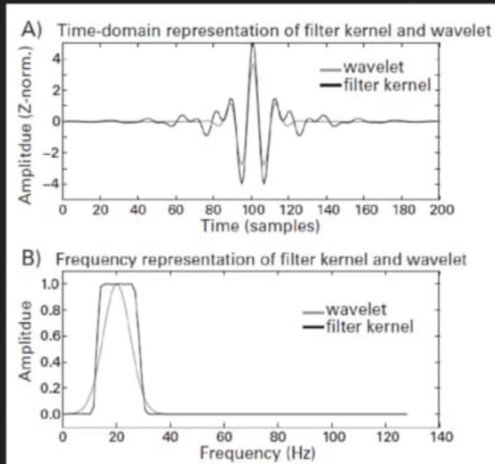
STFFT time-frequency precision trade-off



Fieldtrip website

Comparing wavelet Convolution, Filter Hilbert,
and STFFT :

Wavelet convolution vs. filter-Hilbert



Neural time series analysis — MX Cohen — sincpress.com

Filter-Hilbert

More parameters

Slower computation

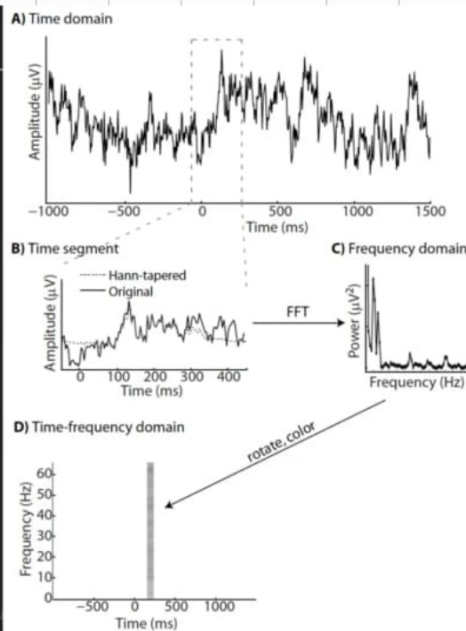
Control over spectral shape

Wavelet conv.

Fewer parameters

Fast computation

Gaussian spectral shape



Wavelets vs. STFFT

STFFT

More parameters

Slower computation

Lower temporal resolution

Wavelet conv.

Fewer parameters

Fast computation

Temporal resolution of data

the Multi taper method :

↳ based on Discrete prolate spheroidal tapers (DPSS)

multi-taper method is designed for data that is not phase locked and not time locked

↳ then they can be lost in trial averaging

↳ Have a method that is more robust to these temporal

jitters essentially by smoothing
over frequency and over time
to allow us to identify

2:08:42



How the multitaper method works

